

A Class for Status and Optional Value

Committee: ISO/IEC JTC1 SC22 WG21 EWG Evolution

Document Number: P0262r1

Title: A Class for Status and Optional Value

Date: 2016-10-15

Authors: Lawrence Crowl, Chris Mysisen

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Audience: EWG Evolution

Abstract

We propose a control helper, `status_value`, that enables functions to return both status and value from a function, which in turn enables the caller to decide when exceptions are appropriate, rather than the callee making that choice.

Contents

[Introduction](#)

[Discussion](#)

[Solution](#)

[Examples](#)

[Wording](#)

[?? Status and Value Objects \[status_value\]](#)

[??1 General \[status_value.general\]](#)

[??2 Header <status_value> synopsis \[status_value.syn\]](#)

[??3 Class template status_value \[status_value.class\]](#)

[??3.1 Constructors \[status_value.ctor\]](#)

[??3.2 Destructor \[status_value.dtor\]](#)

[??3.3 Status observers \[status_value.status\]](#)

[??3.4 State observers \[status_value.state\]](#)

[??3.4 Value observers \[status_value.value\]](#)

[??4 Class template bad_status_value_access \[status_value.bad\]](#)

[Revision History](#)

Introduction

The current proposal for concurrent queues, [P0260r0 C++ Concurrent Queues](#), has two separate waiting pop operations, one with a return value one with an output reference parameter. These functions provide essentially the same semantics, but differ how they signal disappointment. See [P0157r0 Handling Disappointment in C++](#).

```
Value queue::value_pop();
queue_op_status queue::wait_pop(Value&);
```

This design has two consequences.

- A bad status for `value_pop` will result in an exception.
- Use of the `wait_pop` operation requires a `Value` that is essentially default constructible.

Chandler Carruth suggested having a return value that comprised both status and value.

```
something<queue_op_status, Value> queue::pop();
```

The essential point of the design is that *something* may or may not contain a value, and that accessing a non-existent value results in an exception.

This design would normalize the form of the functions, move exception generation to accessing a non-existent value, and enable use of types with no default constructor as elements.

This paper proposes a mechanism to provide this combined status and value. While such a proposal could be folded into the queue proposal alone, the true value of such a mechanism would be its widespread use throughout the library, particularly in concurrent data structures where it is not technically possible to provide separate access functions for status and value.

Therefore, we should either commit to or abandon such a mechanism.

Discussion

The value is clearly related to the class template `optional`, as defined by [N3793 A proposal to add a utility class to represent optional objects \(Revision 5\)](#). The problem could be solved with a tuple of the status and an `optional`. That approach would require more verbose code at each use, which we prefer to avoid.

However, the need is almost directly met by the class template `expected`, as defined by [N4109 A proposal to add a utility class to represent expected monad - Revision 1](#).

In our view, the weakness in the `expected` proposal is that you get either a value or an error, but not both. In the queue proposal, operations return a status, not an error. The distinction is that a queue's inability to deliver a value is often not an error, but simply a normal part of interacting with concurrent objects. For example, `try_pop` can return `queue_op_status::empty`, which clearly does not indicate an error.

The distinction has a design effect when more than one status may be associated with a value. For example, a concurrent queue could provide both a status indicating success with low contention and a status indicating success with high contention. In both cases, the value alone provides less information.

So, we prefer a design in which a status is always provided, but the value is optional. Users of the design will need to define which statuses have values.

Solution

The design is a simpler version of that which appears in [N4109 A proposal to add a utility class to represent expected monad - Revision 1](#). The interface follows the lead of `std::optional` in detailed design as reflected in [N4604 C++17 CD Ballot Document](#). However, there are several definitions in `optional` that do not appear in this proposal. The primary reason is that we propose a control helper, not a data structure. Operations related to persistence and deferred values are not needed.

The working name for the proposed class is `status_value`.

```
template<typename Status, typename Value> class status_value;
```

Construction of a `status_value` can be done with or without a value.

```
status_value::status_value(Status s);
status_value::status_value(Status s, Value&& v);
status_value::status_value(Status s, const Value& v);
```

Construction of `status_value` must include a status.

```
status_value::status_value() = delete;
```

A `status_value` may be moved. A copy operation would make the type unusable for non-copyable contained objects, so we do not provide a copy operation.

```
status_value::status_value(status_value&& sv);
```

They may be queried for status. The design assumes that inlining will remove the cost of returning a reference for cheap copyable types.

```
const Status& status_value::status() const noexcept;
```

They may be queried for whether or not they have a value.

```
bool status_value::has_value() const;
status_value::operator bool() const;
```

They may provide access to their value. If they have no value, an exception of type `bad_status_value_access<Status>`, with the status value passed to the constructor, is thrown.

```
const Value& status_value::value() const;
Value& status_value::value();
const Value& status_value::operator *() const;
Value& status_value::operator *();
```

This design enables moving out of the class by calling `std::move` on the result of the non-const functions.

Examples

The outlined solution changes typical code for the proposed concurrent queue from

```
Value e = q.value_pop();
```

into

```
Value e = q.value_pop().value();
```

and from

```
Value e;  
queue_op_status s = q.wait_pop(e);  
if ( s == queue_op_status::success )  
    do_something_with(e);  
else  
    do_something_else_with(s);
```

into

```
auto sv = q.wait_pop();  
if ( sv.status() == queue_op_status::success )  
    do_something_with(sv.value());  
else  
    do_something_else_with(sv.status());
```

or for handling all statuses that have values

```
if ( auto sv = q.wait_pop() )  
    // Via the implicit conversion to bool, a value is known to be present.  
    do_something_with(*sv);  
else  
    // Via the implicit conversion to bool, a value is known to be absent.  
    do_something_else_with(sv.status());
```

With both sets of changes, the handling of disappointment is decided by the caller, not by the set of operations provided by the queue.

Wording

The proposed wording is as follows.

?? Status and Value Objects [status_value]

Add a new section.

???.1 General [status_value.general]

Add a new section.

This subclause describes class template `status_value` that represents a return status together with an optional object. The class template `status_value` is a control helper that enables deferring to the caller a decision about whether or not an exception is thrown.

???.2 Header <status_value> synopsis [status_value.syn]

```
namespace std {  
  
    // ??.3 class template status_value  
    template <typename Status, typename Value>  
    class status_value;  
  
    // ??.4 class template bad status value access  
    template <typename Status>  
    class bad_status_value_access;
```

```
} // namespace std
```

A program that necessitates the instantiation of template status_value for a reference type is ill-formed.

??3 Class template status_value [status_value.class]

```
template <typename Status, typename Value>
class status_value
{
    // ??3.1 constructors
    status_value() = delete;
    status_value(Status s);
    status_value(Status s, const Value& v);
    status_value(Status s, Value&& v);
    status_value(status_value&& sv) noexcept(see below);

    // ??3.2 destructor
    ~status_value();

    // assignment
    status_value& operator=(const status_value&) = delete;

    // ??3.3 status observers
    const Status& status() const noexcept;

    // ??3.4 state observers
    bool has_value() const noexcept;
    explicit operator bool() const noexcept;

    // ??3.5 value observers
    const Value& value() const &;
    Value& value() &;
    Value&& value() &&;
    const Value&& value() const &&;
    const Value* operator->() const;
    Value* operator->();
    const Value& operator*() const &;
    Value& operator*() &;
    const Value&& operator*() const &&;
    Value&& operator*() &&;
};
```

Any instance of status_value<Status, Value> at any given time either contains a value or does not contain a value. When instance of status_value<Status, Value> contains a value, it means that an object of type Value, referred to as the status_value object's contained value, is allocated within the storage of the status_value object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. The contained value shall be allocated in a region of the status_value storage suitably aligned for the type Value. When an object of type status_value is contextually converted to bool, the conversion returns true if the object contains a value; otherwise the conversion returns false.

Status shall be an object type and shall be copy constructible and destructible.

Value shall be an object type and shall be move constructible and destructible.

??3.1 Constructors [status_value.ctor]

status_value(Status s);

Effects: Constructs a status_value with the Status s and with no Value.

Postcondition: this->has_value() is false.

Throws: Any exception thrown by the selected constructor of Status.

status_value(Status s, const Value& v);

status_value(Status s, Value&& v);

Effects: Constructs a status_value with the Status s and with Value v.

Postcondition: this->has_value() is true.

Throws: Any exception thrown by the selected constructor of Status. Any exception thrown by the selected constructor of Value.

status_value(status_value&& sv) noexcept(see below);

Effects: Constructs a status_value with a copy of sv.status() and a move of sv.value(), if any.

Postcondition: this->has_value() is that of sv->has_value() before construction. sv->has_value() is false.

Throws: Any exception thrown by the selected constructor of Status. Any exception thrown by the selected constructor of Value.

Remarks: The expression inside noexcept is equivalent to is_nothrow_copy_constructible v<Status> && is_nothrow_move_constructible v<Value>.

?.?.3.2 Destructor [status_value.dtor]

~status_value();

Effects: Destroys the object.

?.?.3.3 Status observers [status_value.status]

const Status& status() const noexcept;

Returns: A reference to the status object.

?.?.3.4 State observers [status_value.state]

constexpr bool has_value() const noexcept;

constexpr explicit operator bool() const noexcept;

Returns: True iff *this has a value.

constexpr bool has_value() const noexcept;

constexpr explicit operator bool() const noexcept;

Returns: True iff *this has a value.

?.?.3.4 Value observers [status_value.value]

const Value& value() const &;

Value& value() &;

Value&& value() &&;

const Value&& value() const &&;

const Value* operator->() const;

Value* operator->();

const Value& operator*() const &;

Value& operator*() &;

const Value&& operator*() const &&;

Value&& operator*() &&;

Returns: A reference or pointer to the contained value.

Throws: When !this->has_value(), a bad_status_value_access<Status> constructed with the Status passed to the constructor of status_value.

?.?.4 Class template bad_status_value_access [status_value.bad]

```
template <typename Status>
class bad_status_value_access : public logic_error
{
```

```

    // constructors
    bad_status_value_access() = delete;
    bad_status_value_access(Status s);

    // destructor
    ~bad_status_value_access();

    // status observers
    const Status& status() const noexcept;

};

```

The class `bad_status_value_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of a `status_value` object that does not contain a value.

```
bad_status_value_access(Status s);
```

Effects: Constructs a `bad_status_value_access` with the `Status s`.

Postcondition: `what()` returns and implementation-defined NTBS.

Throws: Any exception thrown by the selected constructor of `Status`.

```
~bad_status_value_access();
```

Effects: Destroys the object.

```
const Status& status() const noexcept;
```

Returns: A reference to the status object.

Revision History

This paper revises [P0262r0](#) as follows.

- Add proposed wording.
- Change signatures to be more in line with `optional`.
- Explicitly delete the assignment operator.
- Add additional value observers.
- Add `noexcept` qualifiers where applicable.
- Change the exception object type from `Status` to `std::bad_status_value_access<Status>`.

P0262r0 revised [N4233](#) as follows.

- Add a reference to [P0157r0 Handling Disappointment in C++](#).
- Add a discussion of a tuple of `status` and `optional`.
- Explicitly avoid copying for `status_value` objects.
- Make the `status` member function return a reference, which enables efficient access to non-trivial status objects.
- Clarify the mechanism for moving out of a `status_value`.
- Change the running example to `value_pop` versus `wait_pop`, which is the essential example of different handling of disappointment.
- Add clarification of the example using implicit conversion to `bool` in a variable declaration in a condition.